CPE 301: Embedded Systems Design

Final Project Part 3

Individual / Group member name: Zach Kabler

---

1. Project Motivation/Introduction

The project I am interested in is a smart blind controller. The motivation behind my project is that during the day I will sometimes leave my blinds open during the hot summer, which results in the sun leaking through and making my room uncomfortable in various ways such as it being too bright or even too hot.

This project would likely be used by someone trying to regulate room conditions assuming they have a window in said room. The type of person to use this would be anyone with a room with a window, who wants a low-cost way to manage how comfortable their room is.

An embedded system is a reasonable solution because it will likely be monitoring sunlight using sensors, and with these sensors, it will create some sort of output to control the blinds.

---

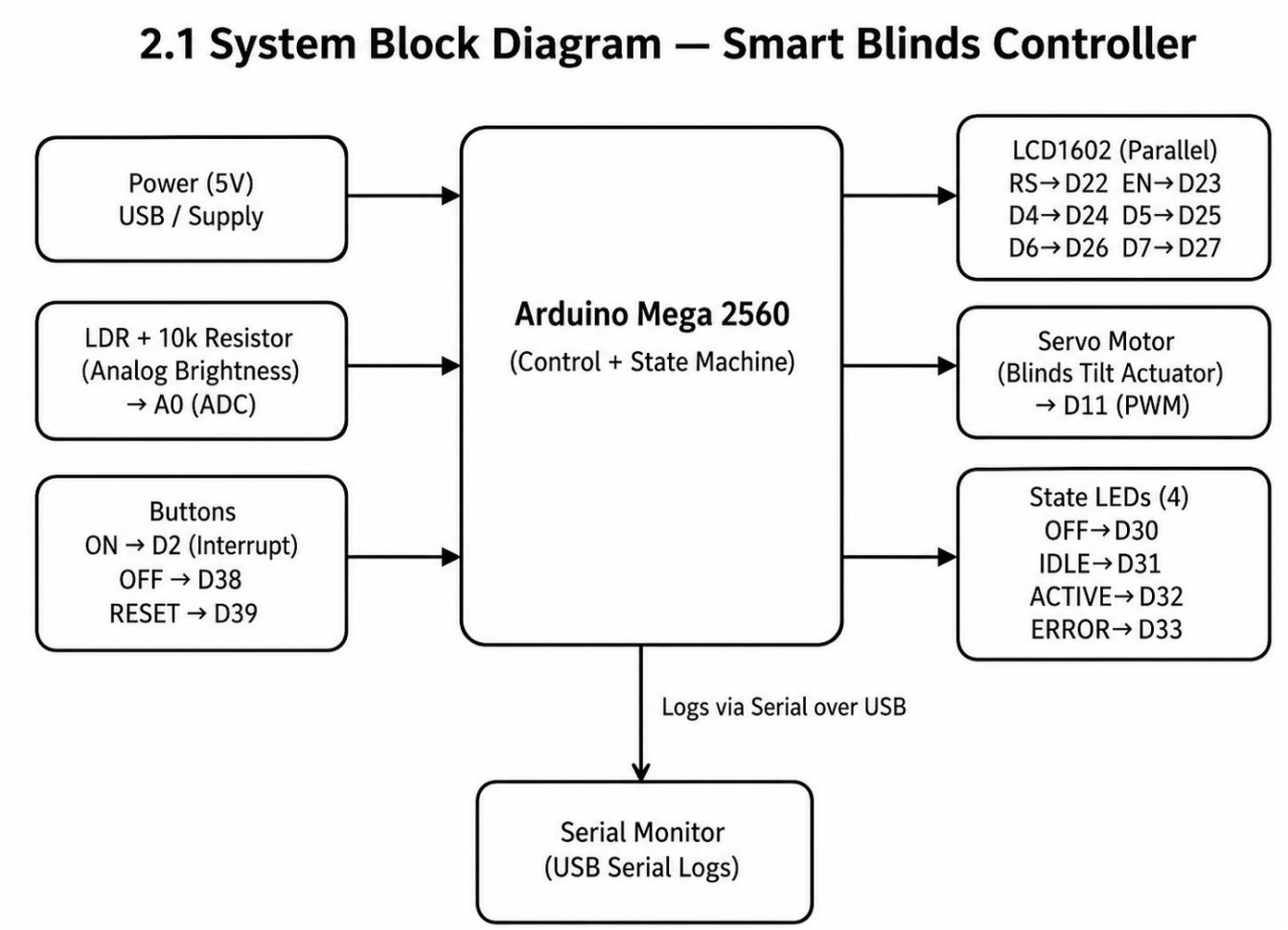
2. Proposed System Overview

The system will be able to detect the brightness of the sun, then based on this information it will be able to adjust the blinds. I will be using a photoresistor(s, servo motor, LEDs, as well as an

LCD display to show the information that the sensors are reading.

The user will be able to turn the system on, turn the system off, and reset the system if an error occurs. The system operates automatically by reading the LDR light sensor and adjusting the servo motor when the light level crosses the threshold. The LCD displays the current state and sensor value, while LEDs indicate OFF, IDLE, ACTIVE, and ERROR states.

2.1 System Block Diagram



---

3. Expected System Behavior

When the system is in the OFF state, the blinds controller is powered but not actively monitoring the light sensor. The OFF LED is on, the actuator is in a safe position, and the LCD displays that the system is off. Pressing the ON button triggers an interrupt and moves the system from OFF to IDLE.

In the IDLE state, the system reads the LDR light sensor using ADC and displays the light value on the

LCD. The IDLE LED is on and the servo keeps the blinds in the normal/open position. If the light value rises above the selected threshold, the system enters the ACTIVE state.

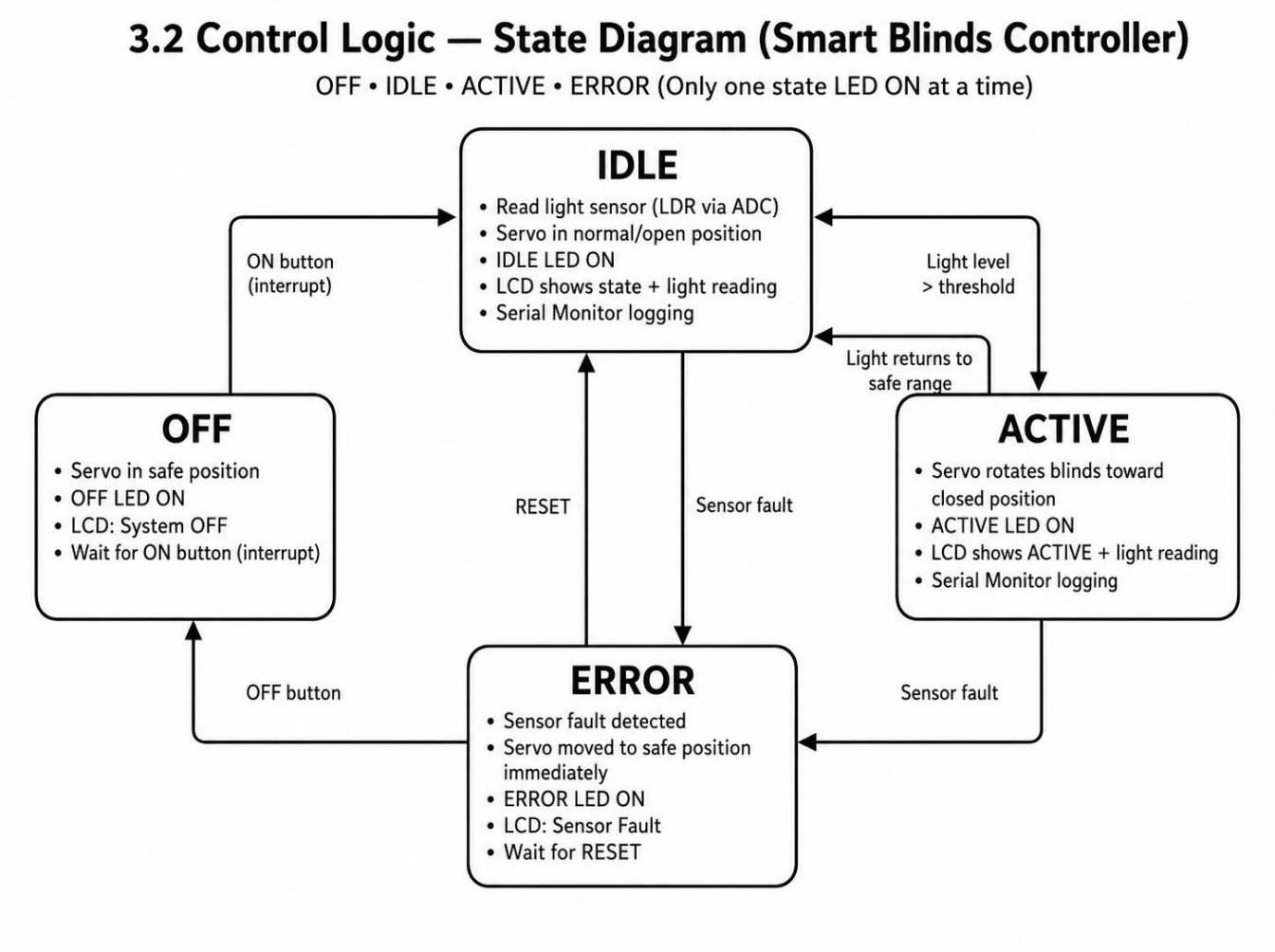
In the ACTIVE state, the system commands the servo motor to rotate the blinds in response to high light. The ACTIVE LED turns on, the LCD displays the ACTIVE state and light value, and the event is logged. When the light value falls back below the safe threshold, the system returns to IDLE.

In the ERROR state, the system detects a sensor fault, such as an invalid ADC reading or disconnected sensor. The servo moves to a safe position, the ERROR LED turns on, and the LCD displays a sensor fault message. Pressing RESET clears the error and returns the system to IDLE.

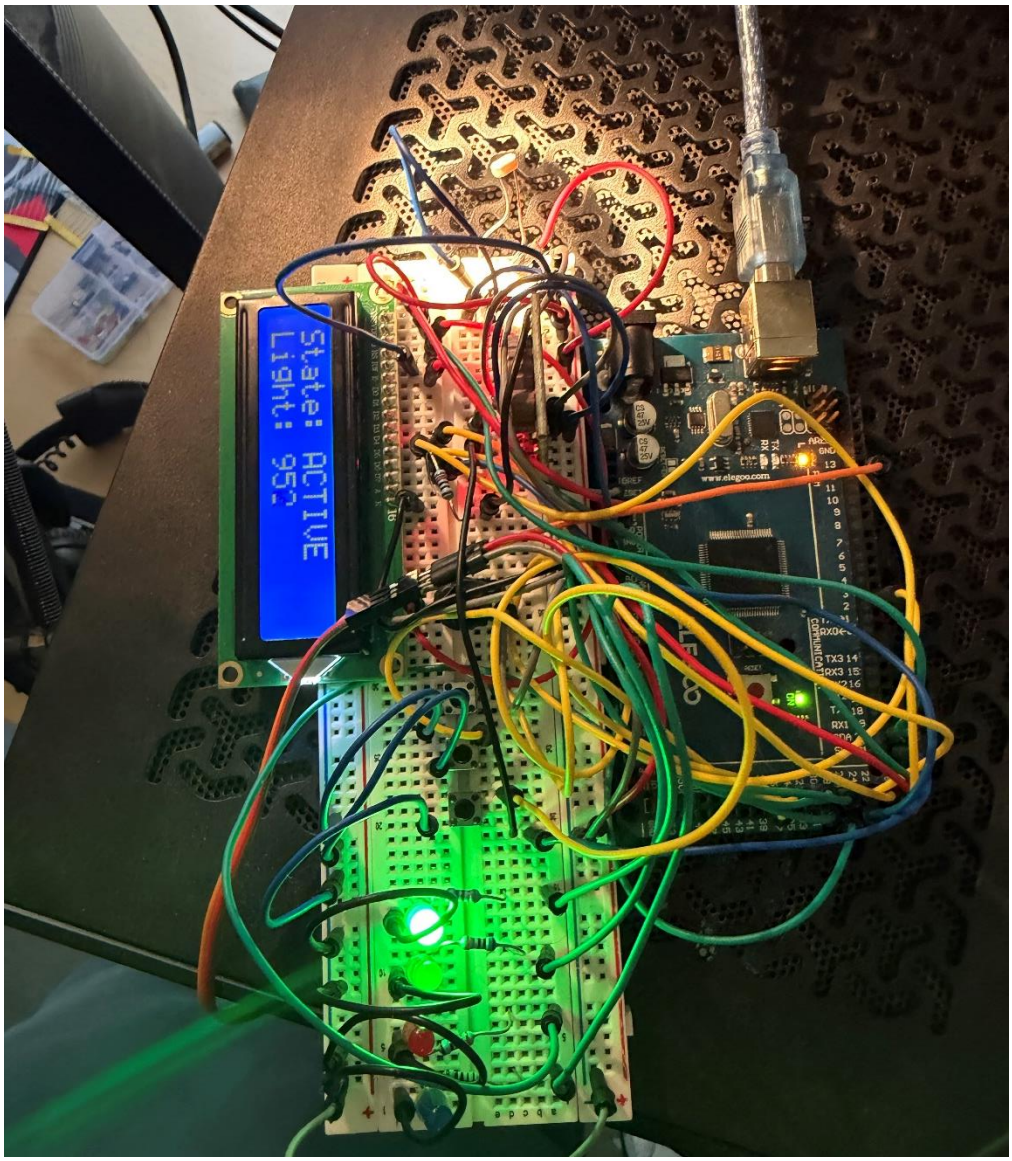
3.1 Module Design

The first module will be an input/sensor module, which will read the LDR using ADC. The second module will be error detection, it will primarily detect if the LDR is stuck or disconnected, if this happens it will output a flag and light up an LED to represent error. The third module will be the control logic, which will implement the required states (on, off, idle, error). The fourth will be the actuator module which will have different functions depending on the state, for example if the system is on, the servo will move to desired angle. Fifth module will be a display module which shows the current state and other values.

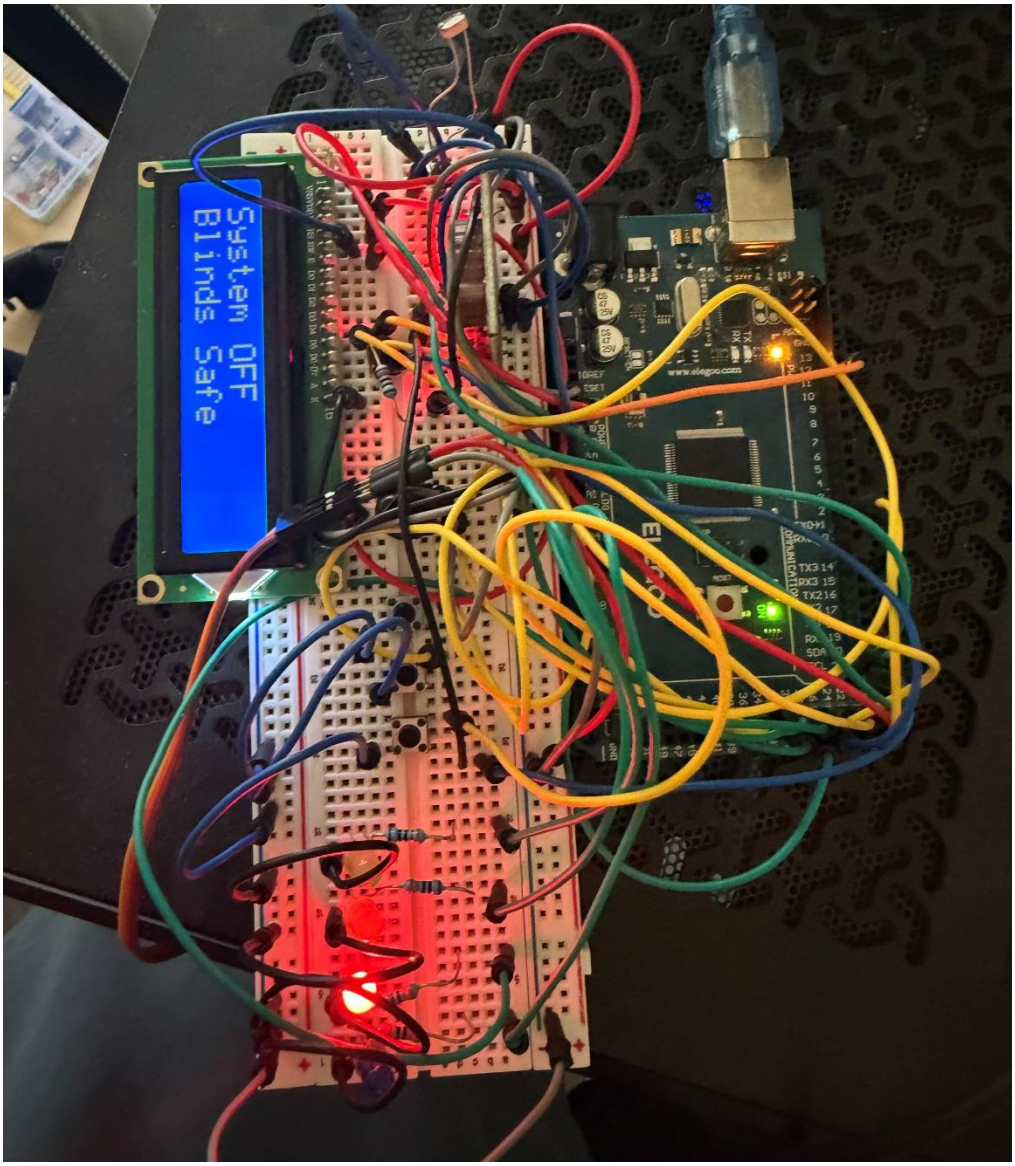
3.2 Control Logic

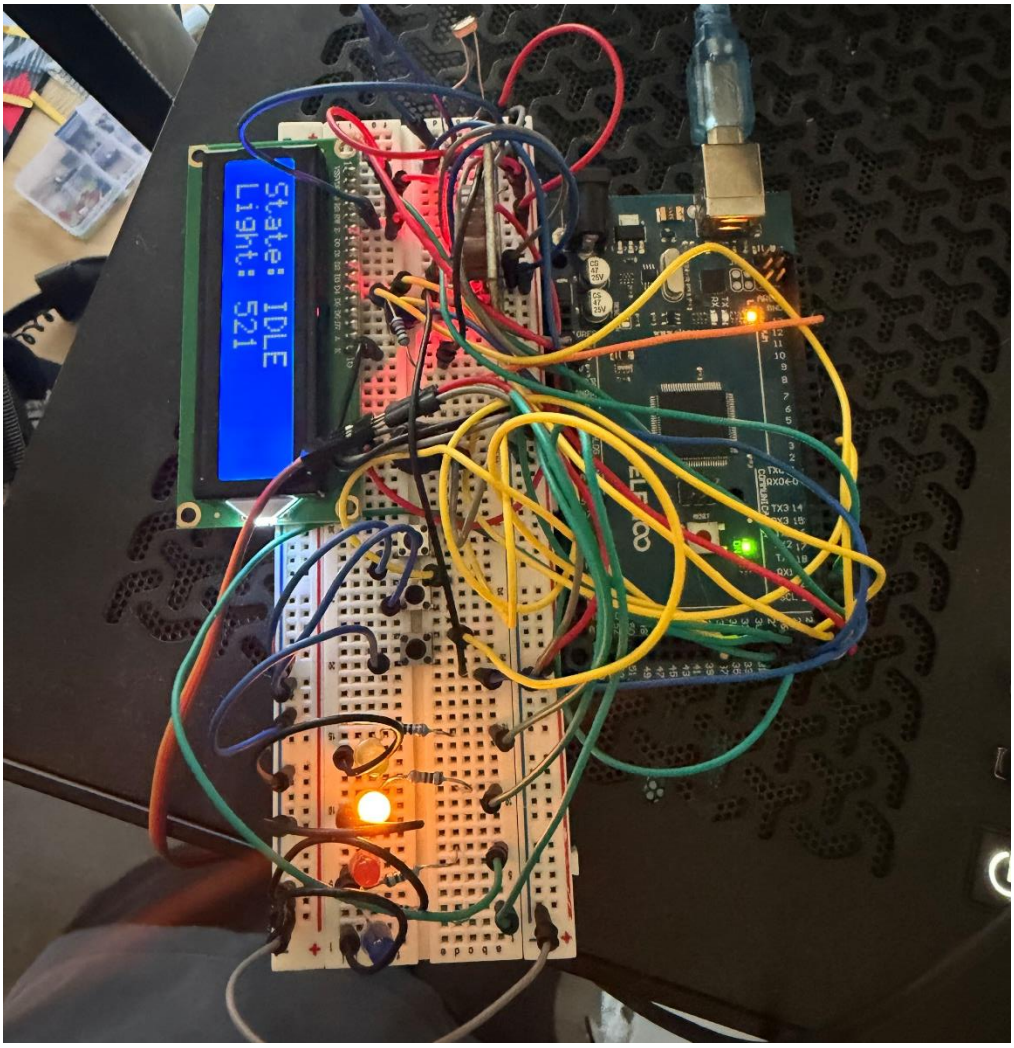

3.2 Control Logic — State Diagram (Smart Blinds Controller)
OFF • IDLE • ACTIVE • ERROR (Only one state LED ON at a time)
IDLE
• Read light sensor (LDR via ADC)
• Servo in normal/open position
• IDLE LED ON
• LCD shows state + light reading
• Serial Monitor logging
ON button (interrupt)
Light level > threshold
Light returns to safe range
OFF
• Servo in safe position
• OFF LED ON
• LCD: System OFF
• Wait for ON button (interrupt)
RESET
Sensor fault
ACTIVE
• Servo rotates blinds toward closed position
• ACTIVE LED ON
• LCD shows ACTIVE + light reading
• Serial Monitor logging
OFF button
ERROR
• Sensor fault detected
• Servo moved to safe position immediately
• ERROR LED ON
• LCD: Sensor Fault
• Wait for RESET
Sensor fault

4. Results

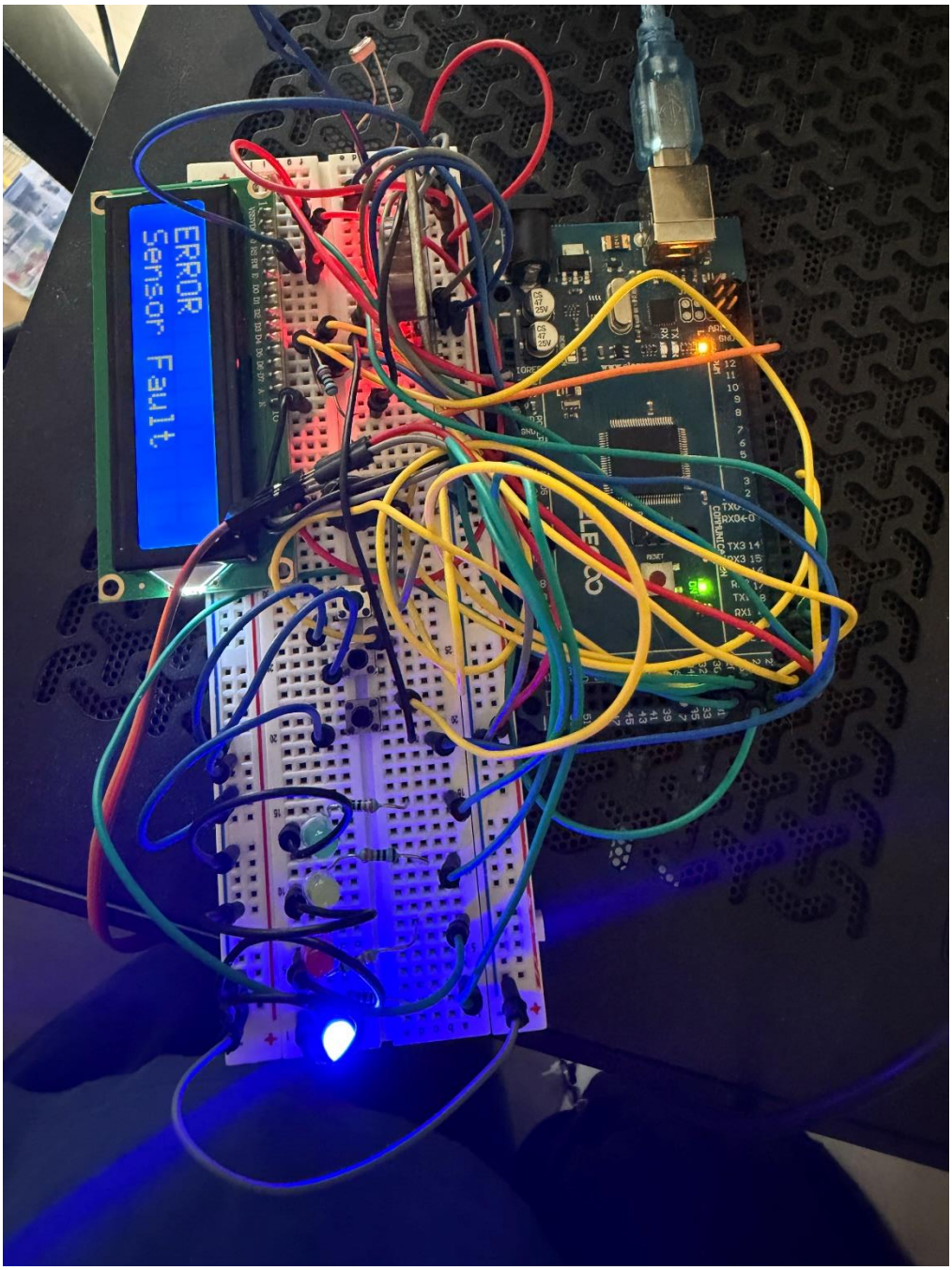


Active (Flashlight on photoresistor)


System OFF
Blinds Safe

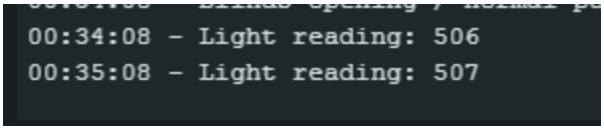


Idle (Below 700 light level)



Error State (Removed photoresistor wire)

```
00:19:58 - IDLE state entered
00:19:58 - Blinds opening / normal position
00:19:59 - ACTIVE state entered
00:19:59 - Blinds closing due to high light
00:20:01 - IDLE state entered
00:20:01 - Blinds opening / normal position
00:20:09 - OFF state entered
00:20:15 - IDLE state entered
00:20:15 - Blinds opening / normal position
00:20:18 - ERROR: Sensor fault
00:20:18 - Blinds moved to safe position
```



1 minute delay

5. Environmental & Energy Impact

The project will minimize energy and reduce waste through reducing the need for a cooler/AC during the hot summer days. Also since the system only reacts based on information, it won't be making constant adjustments, which will save power.

The design will be safe, I will only be using the components from the kit, which are all low voltage components, so the risk is minimal.

The project is affordable, since I will only be using components from the kit, I shouldn't have any outside expenses.

The project promotes sustainability, since it controls the amount of natural sunlight let into a room. Since the system is going to be made from a series of smaller components that work together, and not a giant system, you could simply replace individual components rather than replace a whole system if something were to happen.

The accessibility of the project will be simple, it will have an on/off and reset button, LCD display, and led indicators for feedback.

Assumptions & Constraints

- Only using the course provided kit
- Servo motor strength
- Time constraints

6. Conclusion

In conclusion, the smart blinds controller worked as intended at the prototype level. The system successfully transitioned between the OFF, IDLE, ACTIVE, and ERROR states. The ON button interrupted the system from OFF to IDLE, the LDR light sensor controlled the transition between IDLE and ACTIVE, the RESET button cleared the ERROR state, and the OFF button returned the system to the OFF state. The LCD displayed the current state and light sensor value, the correct state LED turned on for each state, and the servo actuator responded when the light level crossed the threshold.

The main limitation of the project was mechanical. The standard servo motor was able to rotate when commanded, but it did not provide enough total rotation to fully turn the real blind tilt wand, which requires multiple rotations. Because of this, the system demonstrates the control behavior of an automatic blinds controller, but a future version would need a continuous-rotation servo or maybe even a stepper motor to be able to fully rotate the tilt wand. I would also call this a budget limitation because I could have simply ordered a continuous rotation servo, but I wanted to keep under my budget which was no budget, and just the kit by itself.